

RAPPORT DE GESTION DE PROJET

RAG Events MVP

Conception et Déploiement Cloud d'un MVP RAG pour la Recherche
d'Événements Culturels

Samir Belasri

Parcours Data Engineer - OpenClassrooms
Projet 13 - Soutenance de Projet de Fin de Parcours

Mai 2026

Table des matières

1. Introduction	4
2. Analyse des besoins	6
3. Méthodologie et gestion de projet	9
4. Architecture technique	14
5. Infrastructure Cloud AWS	20
6. Déploiement et conteneurisation	24
7. Monitoring et observabilité	27
8. Analyse des risques	30
9. Estimation des coûts	34
10. Résultats et bilan	37
11. Perspectives d'évolution	41
12. Conclusion	44

1. Introduction

1.1 Contexte : l'essor de l'IA générative et des architectures RAG

L'intelligence artificielle générative a connu une accélération sans précédent depuis 2022, portée notamment par la diffusion des grands modèles de langage (LLM). Si ces modèles démontrent des capacités impressionnantes en génération de texte, ils souffrent d'une limitation structurelle majeure : leur connaissance est figée à la date d'entraînement, et ils ne peuvent pas accéder à des données propriétaires ou métier sans mécanisme complémentaire.

C'est précisément pour répondre à cette contrainte que les architectures RAG - Retrieval-Augmented Generation - ont émergé comme un paradigme central de l'ingénierie IA appliquée. Le principe est simple dans sa conception, mais exigeant dans son implémentation : avant de générer une réponse, le système effectue une recherche sémantique dans une base de connaissances vectorielle pour identifier les passages les plus pertinents, puis injecte ce contexte dans le prompt envoyé au LLM. Le résultat est une réponse à la fois précise, contextualisée et ancrée dans des données réelles et actualisables.

Cette architecture répond à un besoin industriel croissant : permettre aux entreprises d'exploiter leurs corpus documentaires internes via des interfaces intelligentes, sans avoir à réentraîner des modèles coûteux. Dans le domaine culturel, cela se traduit concrètement par la possibilité d'interroger en langage naturel des bases d'événements, de concerts, d'expositions ou de spectacles, et d'obtenir des réponses structurées et précises plutôt que de simples listes de résultats.

1.2 Problématique métier

La découverte d'événements culturels reste un problème non résolu à grande échelle. Les plateformes existantes proposent des moteurs de recherche par mots-clés, des filtres par catégorie ou par date, mais aucune ne permet une interrogation véritablement sémantique et conversationnelle. Un utilisateur souhaitant "trouver un concert de jazz intimiste en semaine à Paris avec une ambiance feutrée" se heurte systématiquement aux limites du filtrage classique.

Le projet RAG Events MVP adresse cette problématique en construisant une couche d'intelligence sémantique sur un corpus d'événements culturels. L'objectif n'est pas de reconstruire une plateforme grand public, mais de valider la faisabilité technique d'une API RAG fonctionnelle, déployée dans le cloud, capable d'ingérer des documents, de les vectoriser, et de répondre à des requêtes en langage naturel avec une pertinence mesurable.

1.3 Logique MVP et objectifs

La démarche MVP - Minimum Viable Product - est ici pleinement assumée. Il ne s'agit pas de construire un produit finalisé, mais de démontrer la faisabilité technique d'une architecture RAG déployée dans le cloud sur un périmètre maîtrisé. Cette logique impose des arbitrages clairs : on privilégie la cohérence architecturale sur l'exhaustivité fonctionnelle, la robustesse technique sur la richesse de l'interface, et la documentation sur l'optimisation prématurée.

Les objectifs techniques du projet sont les suivants : développer une API REST performante avec FastAPI exposant les fonctionnalités de recherche et de génération RAG ; construire un index vectoriel FAISS capable d'indexer plus de 1 000 documents et de répondre à des requêtes de similarité sémantique ; conteneuriser l'application avec Docker pour garantir la reproductibilité des déploiements ; déployer l'ensemble sur AWS EC2 dans la région eu-west-3 (Paris) avec une configuration réseau sécurisée ; et documenter l'API via Swagger/OpenAPI pour en faciliter l'intégration future.

Sur le plan métier, le projet vise à démontrer qu'une architecture RAG peut réduire significativement le temps de recherche d'un utilisateur, produire des réponses contextualisées et pertinentes, et constituer une base technique réutilisable pour des services futurs - chatbot, application web ou mobile, intégration dans des systèmes tiers.

2. Analyse des besoins

2.1 Besoins utilisateurs et cas d'usage

L'analyse des besoins a été conduite dès la phase 1 du projet, sur une durée de deux jours selon la roadmap, à travers la définition de personas et de cas d'usage représentatifs. Trois profils d'utilisateurs ont été identifiés : l'utilisateur final souhaitant découvrir des événements culturels via une interface conversationnelle ; le développeur ou intégrateur souhaitant connecter l'API RAG à une application tierce ; et l'administrateur responsable de l'ingestion documentaire et de la maintenance de l'index vectoriel.

Les cas d'usage prioritaires définis pour le MVP sont les suivants. En premier lieu, la recherche sémantique : un client API envoie une requête en langage naturel via l'endpoint `POST /rag/search` et reçoit en retour une liste de documents classés par pertinence sémantique. En second lieu, la génération augmentée : via l'endpoint `POST /rag/answer`, le système récupère les passages les plus pertinents et les utilise comme contexte pour générer une réponse structurée. En troisième lieu, la gestion documentaire : l'endpoint `GET /documents` permet de consulter le corpus indexé, et `POST /rag/build-index` déclenche la (re)construction de l'index vectoriel. Enfin, le monitoring de santé via `GET /health` permet à tout système de supervision de vérifier la disponibilité de l'API.

2.2 Contraintes techniques

L'objectif de performance du MVP est de maintenir des temps de réponse suffisamment faibles pour garantir une expérience utilisateur fluide lors des requêtes de recherche. Cette contrainte oriente le choix vers FAISS, adapté aux recherches vectorielles rapides sur un corpus de taille modérée. La contrainte de scalabilité horizontale future impose une architecture sans état côté applicatif, ce qui justifie la séparation entre l'API FastAPI et le vector store FAISS persistant sur disque.

La contrainte de maintenabilité impose une séparation claire des couches : les endpoints REST ne doivent contenir aucune logique métier, celle-ci étant déléguée aux services (`rag_service.py`, `document_service.py`, `index_service.py`). Cette séparation est visible dans la structure applicative et permet d'évoluer chaque composant indépendamment. La contrainte de portabilité impose la conteneurisation Docker avec une image basée sur Python 3.11, garantissant un comportement identique en local et sur EC2.

2.3 Contraintes cloud et infrastructure

Le projet opère sous une contrainte budgétaire forte : le budget total est fixé à 2 100 euros en phase de build, avec un OPEX mensuel cible de 30 euros. Cette contrainte élimine d'emblée les services AWS managés onéreux tels qu'EKS, RDS ou OpenSearch, et oriente vers une architecture minimaliste centrée sur EC2 t2.micro. Le choix de la région eu-west-3 (Paris) répond à la fois à des impératifs de latence pour les utilisateurs européens et aux exigences de conformité RGPD.

La contrainte de haute disponibilité est intentionnellement limitée au MVP : l'instance EC2 est déployée dans une seule zone de disponibilité, sans load balancer ni auto-scaling group. Cette décision est documentée et tracée dans l'analyse des risques (risque R3 : indisponibilité des services AWS, niveau élevé), avec un plan de mitigation prévu pour les phases ultérieures.

2.4 Contraintes de sécurité

La sécurité réseau est implémentée au niveau du Security Group AWS, avec deux règles entrantes strictement définies : le port 22 (SSH) pour l'administration de l'instance, et le port 8000 (HTTP) pour l'exposition de l'API. Aucun autre port entrant n'est autorisé. Cette configuration

minimaliste respecte le principe du moindre privilège et constitue la première ligne de défense du système.

L'absence d'authentification applicative sur le MVP est documentée comme une limite connue et priorisée en backlog. L'endpoint /config expose des informations de configuration qui, en production, devraient être protégées par un mécanisme JWT ou API Key. Ce choix délibéré permet de simplifier les tests d'intégration lors de la phase MVP tout en documentant clairement la dette technique accumulée.

2.5 Objectifs fonctionnels et techniques synthétiques

Objectifs fonctionnels du MVP :

- Permettre l'interrogation sémantique d'un corpus d'événements culturels en langage naturel
- Exposer une API REST documentée, consommable par tout client HTTP
- Fournir un endpoint de génération de réponses contextualisées
- Assurer la disponibilité et la supervision basique via healthcheck et logs

Objectifs techniques du MVP :

- Indexer plus de 1 000 documents avec embeddings vectoriels dans FAISS
 - Maintenir un temps de réponse API inférieur à 600 ms en conditions normales
 - Atteindre une disponibilité supérieure à 99% sur la durée du projet
 - Déployer l'application dans un conteneur Docker sur EC2 en moins de 2 jours
 - Documenter intégralement l'API via Swagger/OpenAPI 3.1
-

3. Méthodologie et gestion de projet

3.1 Approche Agile adaptée au contexte solo

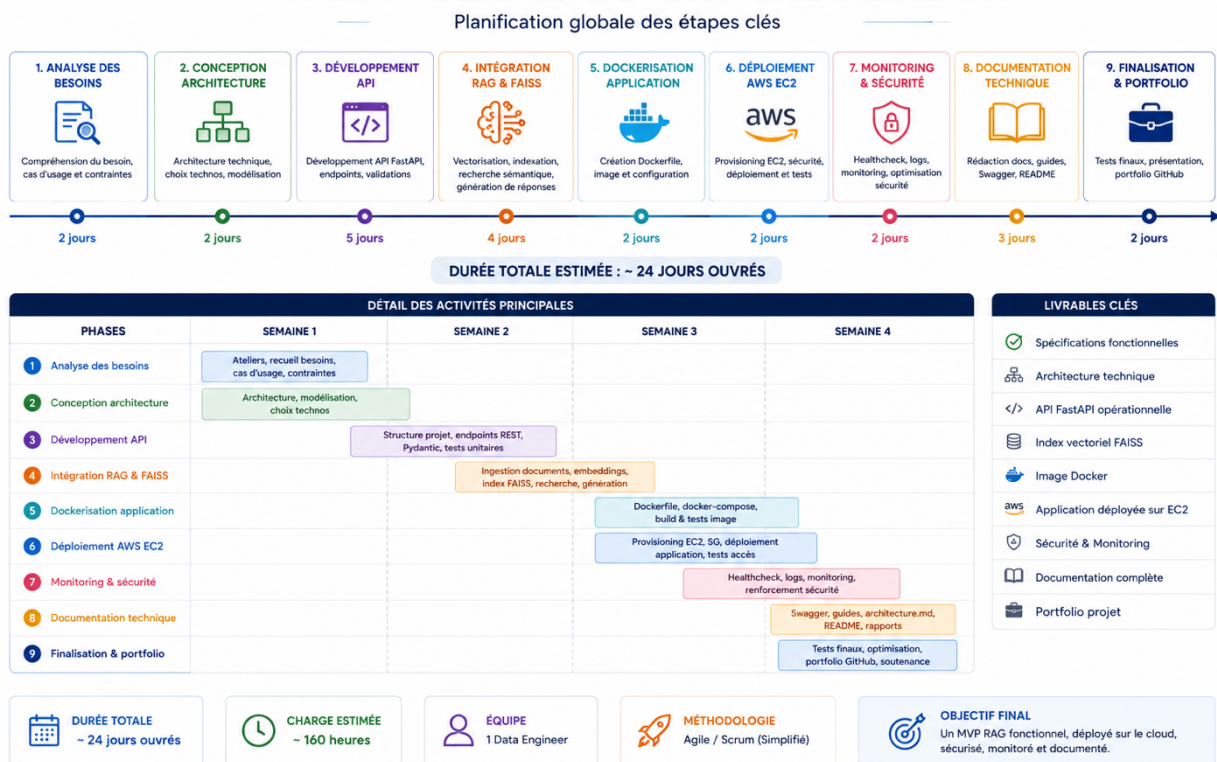
Le projet a été conduit selon une méthodologie Agile simplifiée, inspirée du framework Scrum mais adaptée à la réalité d'un projet conduit par un seul Data Engineer. Cette adaptation est visible dans la roadmap (Image 2) et le planning détaillé (Image 3) : les cérémonies classiques du Scrum (stand-ups, sprint reviews en équipe) ont été remplacées par des points d'étape individuels hebdomadaires et des revues de jalons formalisées.

La logique de développement incrémental est néanmoins pleinement respectée. Chaque phase produit un livrable concret et testable avant que la suivante commence. L'analyse des besoins produit les spécifications fonctionnelles ; la conception architecture produit le schéma validé ; le développement API produit des endpoints fonctionnels avec tests unitaires ; l'intégration RAG & FAISS produit un moteur de recherche opérationnel. Cette dépendance séquentielle est explicitement tracée dans le planning.

3.2 Analyse de la roadmap

La roadmap du projet (Image 2) structure le développement en 9 phases distinctes, pour une durée totale estimée à 24 jours ouvrés et une charge de 160 heures. Cette estimation est cohérente avec la complexité du projet et les technologies impliquées.

IMAGE 2 – ROADMAP DU PROJET RAG EVENTS MVP



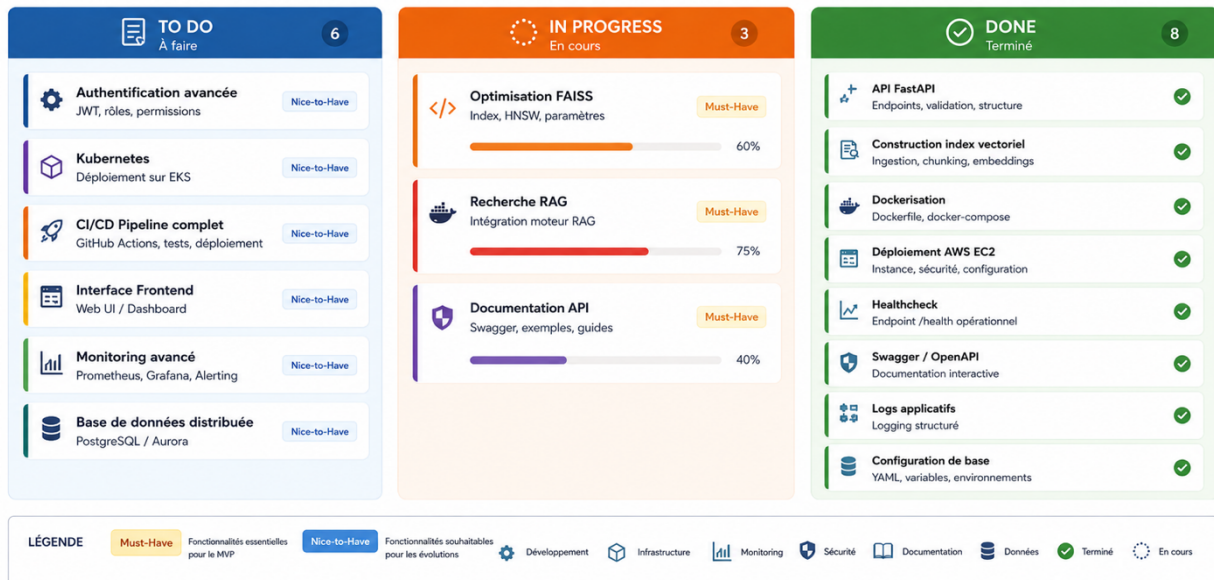
La phase 3 (développement API) est la plus longue avec 5 jours, ce qui est justifié par la complexité de structurer correctement une API FastAPI avec séparation des couches, validation Pydantic et gestion des erreurs. La phase 4 (intégration RAG & FAISS) requiert 4 jours, reflétant la difficulté d'intégrer un pipeline vectoriel complet avec ingestion, embeddings, indexation et génération. Ces deux phases constituent le cœur technique du projet et concentrent l'essentiel de la valeur.

3.3 Backlog Kanban et priorisation

Le backlog Kanban (Image 3) offre une vue claire de l'état d'avancement au moment de la capture. Sur les 17 éléments identifiés, 8 sont en statut DONE, 3 en IN PROGRESS et 6 en TO DO. Cette répartition révèle une maturité technique solide sur les composants fondamentaux du MVP.

IMAGE 3 – BACKLOG KANBAN

Backlog MVP du projet RAG Events MVP



Les 8 éléments terminés couvrent l'ensemble du socle technique indispensable : API FastAPI avec endpoints, validation et structure ; construction de l'index vectoriel FAISS avec ingestion, chunking et embeddings ; dockerisation complète avec Dockerfile et docker-compose ; déploiement AWS EC2 avec configuration de sécurité ; endpoint healthcheck opérationnel ; Swagger/OpenAPI pour la documentation interactive ; logs applicatifs structurés ; et configuration YAML avec gestion des variables d'environnement.

Les 3 éléments en cours de traitement sont classés Must-Have pour le MVP : l'optimisation FAISS (index HNSW, paramètres) à 60% d'avancement ; la recherche RAG avec intégration complète du moteur à 75% ; et la documentation API Swagger à 40%. Ces trois chantiers sont en progression directe vers la finalisation.

Les 6 éléments en TO DO sont tous tagués Nice-to-Have, ce qui confirme que la priorisation MVP est correctement appliquée : authentification avancée JWT, Kubernetes sur EKS, CI/CD avec GitHub Actions, interface frontend, monitoring avancé Prometheus/Grafana, et base de données distribuée PostgreSQL/Aurora. Aucun de ces éléments ne bloque la livraison du MVP.

3.4 Planning détaillé et jalons

Le planning de 12 semaines (Image 3) formalise 9 jalons clés avec leur semaine cible de livraison. L'analyse du Gantt révèle deux zones de parallélisme stratégique : les phases de développement API et d'intégration RAG se chevauchent partiellement (S5-S6), permettant de valider les endpoints REST pendant que l'intégration FAISS progresse. De même, la dockerisation démarre en S7 pendant que le RAG finalise, optimisant l'utilisation du temps.

Les hypothèses et contraintes listées dans le planning sont réalistes et honnêtes : accès continu aux services AWS, disponibilité des jeux de données, respect des délais par phase, et

environnement stable pour les tests. Ces hypothèses ont été vérifiées avant le lancement et constituent les prérequis du projet.

La gestion des risques intégrée au planning identifie quatre risques prioritaires : le retard d'intégration RAG (impact élevé, probabilité moyenne), les problèmes de performance (impact moyen, probabilité moyenne), l'indisponibilité AWS (impact élevé, probabilité faible), et la sécurité insuffisante (impact élevé, probabilité faible). Ces risques sont traités en détail dans la section 8.

3.5 KPI de gestion de projet

Le tableau de bord KPI (Image 5) fournit une vue quantitative de l'avancement au moment de la capture. L'avancement global est de 68% avec 24 tâches terminées sur 35, 8 jours restants sur les 24 jours ouvrés planifiés. La charge consommée est de 108 heures sur 160 estimées, soit un taux de consommation de 67,5% parfaitement aligné avec le taux d'avancement. Le budget consommé est de 1 450 euros sur 2 100, soit 69%, ce qui est cohérent avec l'état d'avancement.

Le burndown chart inclus dans le tableau de bord montre une progression réelle (courbe verte) légèrement en deçà de l'objectif (courbe rouge pointillée), avec 52 heures restantes à faire. Cette légère déviation est normale et sous contrôle, confirmée par un taux d'avancement des actions de mitigation à 75%.

4. Architecture technique

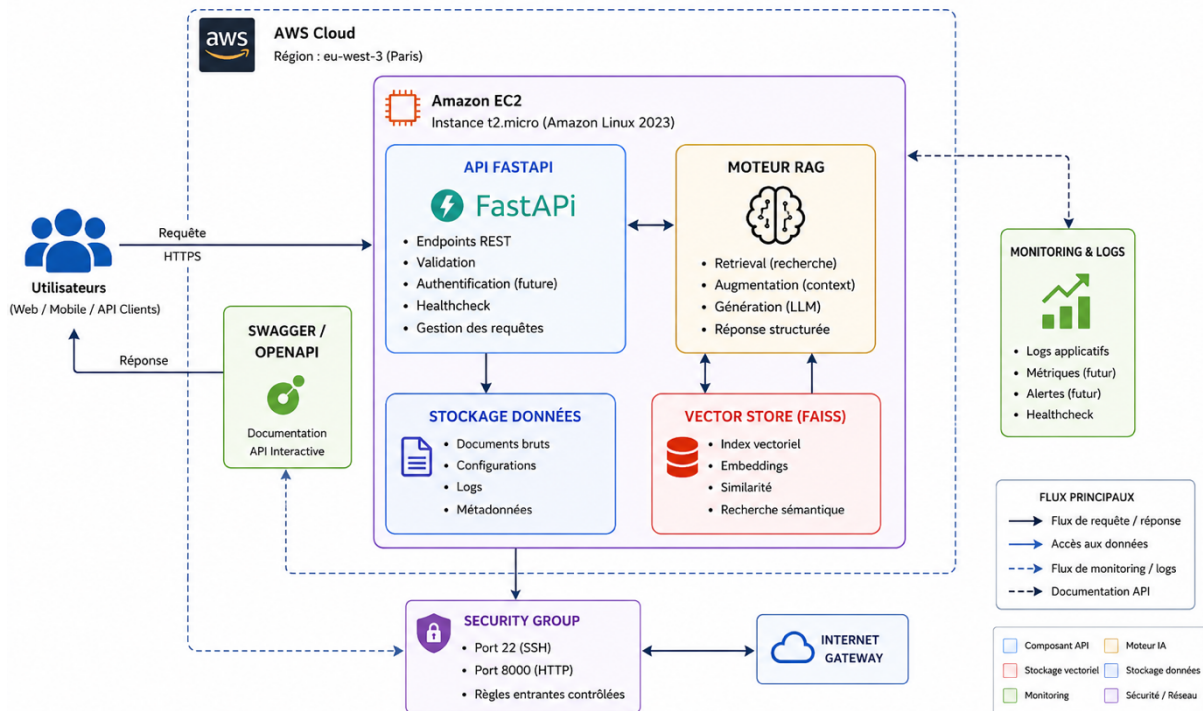
4.1 Vue d'ensemble de l'architecture

Le schéma d'architecture globale (Image 1) présente un système modulaire organisé en couches fonctionnelles distinctes, toutes hébergées sur une instance EC2 t2.micro dans la région AWS eu-west-3 (Paris). Cette architecture monolithique conteneurisée est un choix délibéré pour le MVP : elle simplifie le déploiement et la supervision tout en préservant une séparation logique des composants qui facilitera la décomposition en microservices si nécessaire.

L'architecture se compose de six blocs fonctionnels principaux : l'API FastAPI comme point d'entrée unique, le moteur RAG comme cerveau du système, le vector store FAISS comme base de connaissances sémantique, le stockage de données pour les fichiers bruts et les configurations, le bloc Swagger/OpenAPI pour la documentation interactive, et le bloc Monitoring & Logs pour l'observabilité. Ces six blocs communiquent selon des flux précisément définis dans le schéma : requêtes/réponses HTTP, accès aux données, flux de monitoring et documentation API.

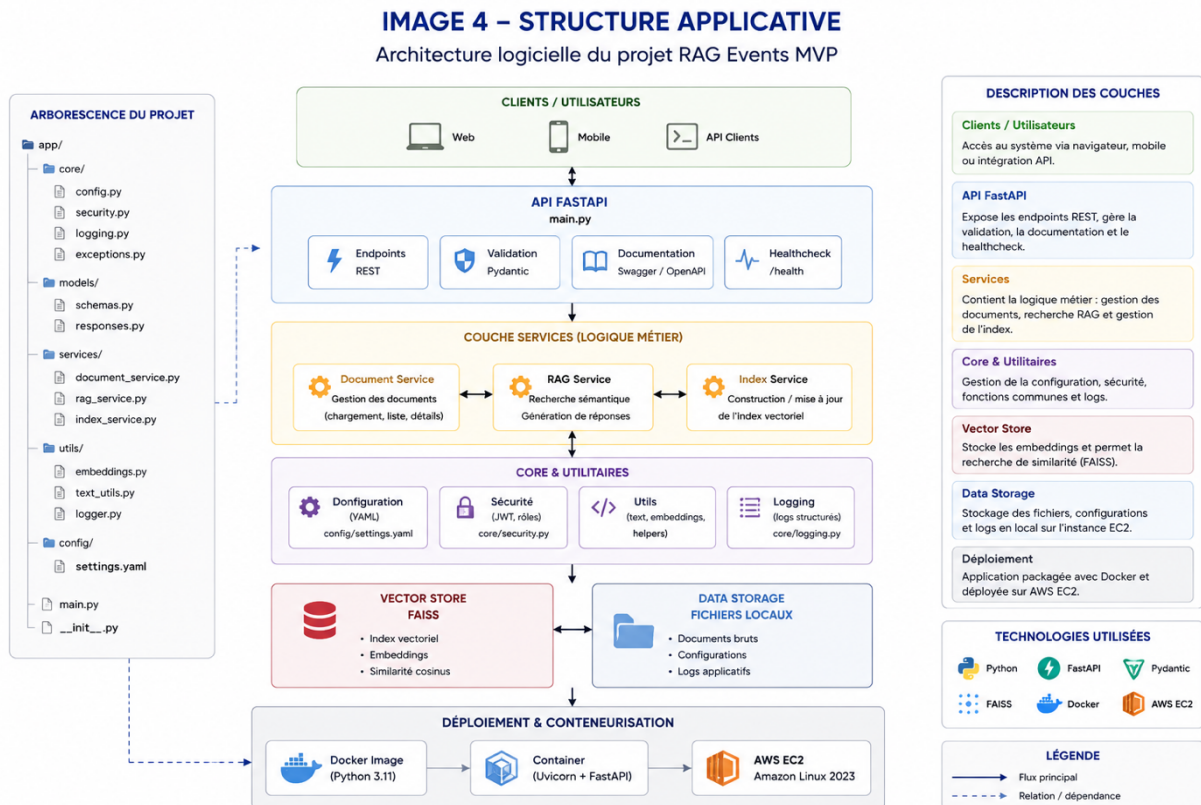
IMAGE 1 – SCHÉMA D'ARCHITECTURE GLOBALE

Architecture Cloud du système RAG Events MVP



4.2 Structure applicative et organisation du code

La structure applicative (Image 4) révèle une organisation en couches de haute qualité technique, conforme aux bonnes pratiques de développement Python et FastAPI. L'arborescence du projet est organisée selon un modèle en couches strictement hiérarchisées.



La couche présentation est incarnée par FastAPI (main.py), qui expose les endpoints REST, gère la validation Pydantic, génère automatiquement la documentation Swagger/OpenAPI et implémente le healthcheck. Cette couche ne contient aucune logique métier - elle délègue intégralement aux services.

La couche services (app/services/) contient les trois services métier fondamentaux : document_service.py pour la gestion des documents (chargement, liste, détails), rag_service.py pour la recherche sémantique et la génération de réponses, et index_service.py pour la construction et la mise à jour de l'index vectoriel. Cette séparation est fondamentale : elle permet de tester chaque service isolément, de les faire évoluer indépendamment et de les substituer si nécessaire.

La couche utilitaires et core (app/core/ et app/utils/) regroupe les fonctions transverses : configuration YAML (settings.yaml), gestion de la sécurité, fonctions utilitaires pour les textes et les embeddings, et le logger structuré. Cette couche ne dépend d'aucune autre couche applicative, garantissant l'absence de dépendances circulaires.

La couche données comprend deux sous-systèmes : le vector store FAISS (vector_store/faiss.index) qui stocke les embeddings et permet la recherche par similarité cosinus, et le stockage de données fichiers locaux (data/) pour les documents bruts, les configurations et les logs applicatifs. La persistance de l'index FAISS sur disque est critique : elle garantit que l'index n'est pas reconstruit à chaque redémarrage du conteneur.

4.3 Pipeline RAG : analyse technique détaillée

Le pipeline RAG est le composant central du projet. Il implémente les trois étapes fondamentales de l'architecture Retrieval-Augmented Generation selon un flux précis et maîtrisé.

Étape 1 - Ingestion et vectorisation

Lors de la construction de l'index (POST /rag/build-index), les documents bruts sont chargés depuis le stockage de données, découpés en chunks de taille optimisée (chunking), puis transformés en vecteurs d'embeddings par un modèle de représentation sémantique. Ces embeddings sont ensuite indexés dans FAISS, qui construit une structure de données optimisée pour la recherche par similarité. L'index est persisté sur disque sous la forme d'un fichier faiss.index.

Étape 2 - Retrieval sémantique

Lors d'une requête de recherche (POST /rag/search), la requête utilisateur est transformée en vecteur d'embedding par le même modèle utilisé lors de l'ingestion - cette cohérence est fondamentale pour la qualité des résultats. FAISS effectue ensuite une recherche des k plus proches voisins par similarité cosinus dans l'espace vectoriel. Les documents retournés sont classés par score de pertinence. Le tableau de bord KPI indique un temps de réponse moyen de 1,2 seconde pour cette étape, avec une disponibilité de 98,9%.

Étape 3 - Augmentation et génération

Pour l'endpoint POST /rag/answer, les passages les plus pertinents récupérés lors de l'étape 2 sont injectés comme contexte dans un prompt structuré envoyé au LLM. Le moteur RAG assemble la réponse finale en combinant le contexte documentaire et les capacités de génération du modèle. La réponse est structurée et renvoyée au client via l'API.

4.4 Analyse des endpoints Swagger

La capture Swagger (Image 15) valide la mise en production effective de l'API à l'adresse <http://13.39.22.155:8000/docs>, avec la mention explicite RAG Events MVP version 1.0.0 / OAS 3.1. Cette capture est une preuve technique irréfutable du déploiement cloud opérationnel.

L'API expose 8 endpoints documentés, organisés en deux groupes fonctionnels. Les endpoints d'infrastructure comprennent : GET / (Home, point d'entrée), GET /health (Health Check, supervision), GET /config (configuration courante), et GET /documents (liste du corpus indexé). Les endpoints RAG comprennent : POST /rag/build-index (construction de l'index vectoriel), GET /rag/status (état du système RAG), POST /rag/search (recherche sémantique), et POST /rag/answer (génération de réponse augmentée).

Endpoint	Méthode	Fonction	Statut
GET /	GET	Point d'entrée, informations API	Opérationnel
GET /health	GET	Health Check - supervision	Opérationnel
GET /config	GET	Configuration courante	Opérationnel
GET /documents	GET	Liste des documents indexés	Opérationnel
POST /rag/build-index	POST	Construction index FAISS	Opérationnel
GET /rag/status	GET	Statut du moteur RAG	Opérationnel
POST /rag/search	POST	Recherche sémantique vectorielle	Opérationnel
POST /rag/answer	POST	Génération de réponse RAG	Opérationnel

La validation Pydantic implémentée sur les endpoints POST garantit la cohérence des données entrantes et fournit des messages d'erreur structurés en cas de requête malformée. Cette validation au niveau de l'API évite la propagation d'erreurs vers les couches inférieures.

4.5 Modularité et maintenabilité

L'architecture applicative présente plusieurs propriétés favorables à la maintenabilité à long terme. La configuration externalisée dans `config/settings.yaml` permet de modifier les paramètres sans toucher au code applicatif. Les fichiers `.env` et `.env.example` garantissent la portabilité entre environnements. La présence d'un répertoire `tests/` dans la structure confirme l'intégration des tests dès la conception. Les logs structurés dans `logs/app.log` et le répertoire `monitoring/` anticipent les besoins d'observabilité.

5. Infrastructure Cloud AWS

5.1 Vue d'ensemble de l'infrastructure AWS

L'infrastructure AWS du projet est construite sur un modèle délibérément minimaliste, conforme aux contraintes budgétaires du MVP. Elle repose sur une instance EC2 t2.micro dans la région eu-west-3 (Paris), opérant dans un VPC dédié avec un Security Group configuré au strict minimum fonctionnel. Ce choix est la résultante directe de l'analyse coût/bénéfice réalisée en phase de conception : un t2.micro à 10 euros/mois suffit largement pour héberger un conteneur Docker FastAPI/FAISS sous une charge de test, et toute surcouche d'infrastructure (ELB, ASG, RDS) aurait multiplié les coûts sans apporter de valeur à ce stade.

5.2 Instance EC2 : configuration et justification

La capture de la console AWS (Image 2) documente précisément l'état de l'instance au moment du déploiement. L'instance p13-rag-server (ID i-0c6f2453593615987) est de type t2.micro, tournant sous Amazon Linux 2023 dans la région eu-west-3. Son adresse IP publique est 13.38.94.16, et son IP privée est 172.31.43.229 dans le VPC vpc-0d4ef4c09b3c4798e. Le statut "En cours d'exécution" confirme l'opérationnalité du déploiement.

Le choix du t2.micro est justifié par le modèle de charge du MVP : l'application FastAPI/Uvicorn est un processus léger en mémoire, FAISS opère sur un index de taille modérée (quelques centaines de Mo pour 1 000+ documents), et la charge de test ne génère pas de concurrence élevée. Le modèle burstable du t2.micro, qui accumule des crédits CPU lors des périodes inactives et les consomme lors des pics, est adapté à ce profil de charge intermittent.

Amazon Linux 2023 a été sélectionné comme système d'exploitation pour plusieurs raisons techniques : sa certification AWS garantit une optimisation native pour EC2 (drivers, patches de sécurité, intégration avec les services AWS) ; sa compatibilité avec les packages RPM facilite l'installation de Docker via dnf ; et son cycle de support jusqu'en 2028 garantit la stabilité de la base.

5.3 Sécurité réseau : Security Group

La configuration du Security Group est visible dans les captures de déploiement et dans le schéma d'architecture. Deux règles entrantes seulement sont définies : le port 22 (SSH) pour les connexions administratives depuis des adresses autorisées, et le port 8000 (HTTP) pour l'accès à l'API FastAPI. Cette configuration respecte strictement le principe du moindre privilège - aucun autre port n'est exposé au trafic entrant.

L'absence de règles HTTPS (port 443) est une limite documentée du MVP : l'API est exposée en HTTP non chiffré, ce qui est acceptable en environnement de test mais inacceptable en production. La migration vers HTTPS avec un certificat SSL (via AWS Certificate Manager et un Application Load Balancer, ou via Nginx en reverse proxy sur l'instance) est identifiée dans les perspectives d'évolution.

Note de sécurité - MVP vs Production

La configuration réseau actuelle (HTTP port 8000, pas d'authentification applicative) est volontairement simplifiée pour le MVP. En production, trois mesures minimales seraient requises : (1) HTTPS avec TLS via un certificat ACM ou Let's Encrypt, (2) authentification JWT ou API Key sur les endpoints sensibles, (3) restriction des sources SSH à des IPs spécifiques plutôt qu'à 0.0.0.0/0.

5.4 VPC et architecture réseau

L'instance est déployée dans un VPC dont l'identifiant `vpc-0d4ef4c09b3c4798e` est visible dans la console AWS. Le VPC fournit un réseau privé isolé dans lequel opère l'instance, avec un Internet Gateway permettant les communications entrantes et sortantes. L'adresse IP publique `13.38.94.16` est allouée dynamiquement à l'instance, ce qui constitue une fragilité pour le MVP : en cas de redémarrage de l'instance, l'IP changera. La migration vers une Elastic IP adresse cette limitation et est documentée dans les évolutions.

5.5 Choix AWS : justification stratégique

Le choix d'AWS comme plateforme cloud est motivé par plusieurs facteurs convergents. La maturité de la plateforme EC2, lancée en 2006 et utilisée par des millions d'entreprises, garantit une stabilité et une documentation exhaustive sans équivalent. L'écosystème de services complémentaires (CloudWatch, IAM, S3, RDS, EKS) offre une trajectoire d'évolution claire et maîtrisée : chaque composant du MVP peut être remplacé par un service AWS managé lorsque la charge le justifie. Le choix d'une instance EC2 unique permet de conserver une infrastructure légère et facilement administrable dans le cadre d'un projet de démonstration technique. Cette approche simplifie le déploiement, la supervision et les opérations de maintenance tout en restant cohérente avec le périmètre du MVP.

La région eu-west-3 (Paris) n'est pas un choix anodin : elle garantit la conformité RGPD en maintenant les données en Europe, minimise la latence pour les utilisateurs français et européens, et est correctement intégrée à la grille tarifaire standard d'AWS. Le choix d'AWS permet de s'appuyer sur une infrastructure cloud reconnue et largement utilisée dans les environnements professionnels.

5.6 Pourquoi FastAPI et Docker

FastAPI a été retenu comme framework API pour trois raisons techniques distinctes. Sa performance native, basée sur Starlette et Pydantic, le positionne systématiquement parmi les frameworks Python les plus rapides dans les benchmarks indépendants. Sa génération automatique de documentation OpenAPI/Swagger, directement à partir des annotations de type Python, supprime le travail redondant de maintien d'une documentation séparée. Enfin, son support natif de l'asynchronisme (`async/await`) prépare l'application aux scénarios de charge élevée.

Docker résout le problème fondamental de la reproductibilité des déploiements. L'image Docker basée sur Python 3.11 embarque toutes les dépendances (`requirements.txt`), garantissant que l'application se comporte identiquement sur le MacBook du développeur, en CI/CD et sur l'instance EC2. Le fichier `docker-compose.yml` visible dans la structure du projet simplifie le démarrage multi-conteneurs et sera la base de l'évolution vers Docker Compose sur EC2 ou vers Kubernetes.

6. Déploiement et conteneurisation

6.1 Workflow de déploiement complet

Le déploiement de l'application sur AWS EC2 a suivi un processus en plusieurs étapes documentées par les captures SSH et Docker. Le workflow complet, reconstituable depuis les captures (Images 1 et 13), se déroule en six étapes séquentielles.

Première étape : provisioning de l'instance EC2. L'instance t2.micro sous Amazon Linux 2023 est créée via la console AWS dans la région eu-west-3, avec le Security Group configuré (ports 22 et 8000). La paire de clés SSH p13-key.pem est générée et sauvegardée localement.

Deuxième étape : connexion SSH depuis le poste de développement. La capture (Image 14) montre le workflow complet de préparation de la clé : création du répertoire ~/.ssh/, copie de la clé PEM, application des permissions restreintes (chmod 400), vérification de la clé (ls -l). La connexion SSH est établie via la commande `ssh -i ~/.ssh/p13-key.pem ec2-user@13.38.94.16`. La bannière Amazon Linux 2023 confirme la connexion réussie.

Troisième étape : configuration du système sur l'instance. La capture (Image 1) montre la séquence d'installation de Docker sur Amazon Linux 2023 via dnf : vérification et installation de 11 paquets (container-selinux, containerd, docker, iptables, libcgrou, libnetfilter, libnfnetlink, libnftnl, pigz, runc). L'activation et le démarrage du service Docker (`sudo systemctl start docker`, `sudo systemctl enable docker`) puis l'ajout de l'utilisateur ec2-user au groupe docker confirment la configuration complète.

Quatrième étape : vérification de Docker sur l'instance. La capture (Image 13) montre la connexion SSH réussie avec la bannière Amazon Linux 2023, suivie des commandes de vérification : `docker --version` retourne Docker version 25.0.14, build 0bab007 et `docker ps` affiche la liste des conteneurs (vide au moment de la capture, avant le déploiement de l'application).

Cinquième étape : déploiement de l'application. Le code source est transféré sur l'instance (via git clone ou scp), l'image Docker est construite localement ou pullée depuis un registre, et le conteneur est lancé via docker-compose ou docker run avec les variables d'environnement appropriées et le binding du port 8000.

Sixième étape : validation du déploiement. La capture Swagger (Image 15) montre l'interface Swagger UI accessible à l'adresse `http://13.39.22.155:8000/docs` depuis un navigateur externe, confirmant que l'API est correctement exposée sur le port 8000 et accessible depuis Internet via l'Internet Gateway AWS.

6.2 Image Docker et conteneurisation

L'image Docker est construite à partir d'un Dockerfile Python 3.11 dont la structure est visible dans le répertoire docker/ de l'arborescence. L'approche de conteneurisation choisie encapsule l'intégralité de la stack applicative dans un seul conteneur : FastAPI, Uvicorn, FAISS et toutes les dépendances Python. Cette décision simplifie le déploiement initial au prix d'une densification du conteneur.

Uvicorn est utilisé comme serveur ASGI pour exécuter l'application FastAPI. Ce choix est standard dans l'écosystème FastAPI : Uvicorn combine des performances élevées (C-extensions pour le parsing HTTP) avec une simplicité de configuration. Pour le MVP, un processus Uvicorn unique suffit. Pour la production, une configuration multi-workers (`uvicorn --workers 4`) ou Gunicorn comme process manager serait recommandée.

Le fichier docker-compose.yml visible dans l'arborescence du projet orchestre le démarrage du service applicatif avec ses dépendances de configuration : variables d'environnement depuis .env, montage du volume vector_store/ pour la persistance de l'index FAISS, et exposition du port

8000. Cette configuration docker-compose est à la fois l'outil de développement local et la base du déploiement EC2.

6.3 Tests et validation du déploiement

La validation du déploiement a été conduite à trois niveaux complémentaires. Au niveau infrastructure : vérification de la connectivité SSH sur le port 22 depuis le poste de développement ; vérification de l'accessibilité HTTP sur le port 8000 depuis un navigateur externe ; vérification du statut de l'instance EC2 dans la console AWS (statut 'En cours d'exécution'). Au niveau applicatif : accès à l'interface Swagger UI sur /docs confirmant le démarrage correct de FastAPI ; appel de l'endpoint GET /health retournant un statut 200 ; appel de GET /config retournant la configuration courante. Au niveau fonctionnel : test des endpoints RAG avec un corpus de données représentatif, mesure des temps de réponse, et vérification de la pertinence des résultats de recherche.

Le tableau de bord KPI indique 100% de taux de réussite sur les requêtes de test, avec un temps de réponse moyen de l'API FastAPI à 180 ms et de l'index FAISS à 320 ms. Ces chiffres valident la performance du système dans des conditions de charge légère, cohérentes avec un MVP.

7. Monitoring et observabilité

7.1 Observabilité du MVP : état actuel

L'observabilité du système au stade MVP repose sur trois mécanismes complémentaires : les logs applicatifs structurés, l'endpoint de healthcheck, et les métriques de performance exposées. Cette couche d'observabilité basique est fonctionnelle et opérationnelle, comme le confirme le backlog Kanban (statut DONE pour 'Logs applicatifs' et 'Healthcheck').

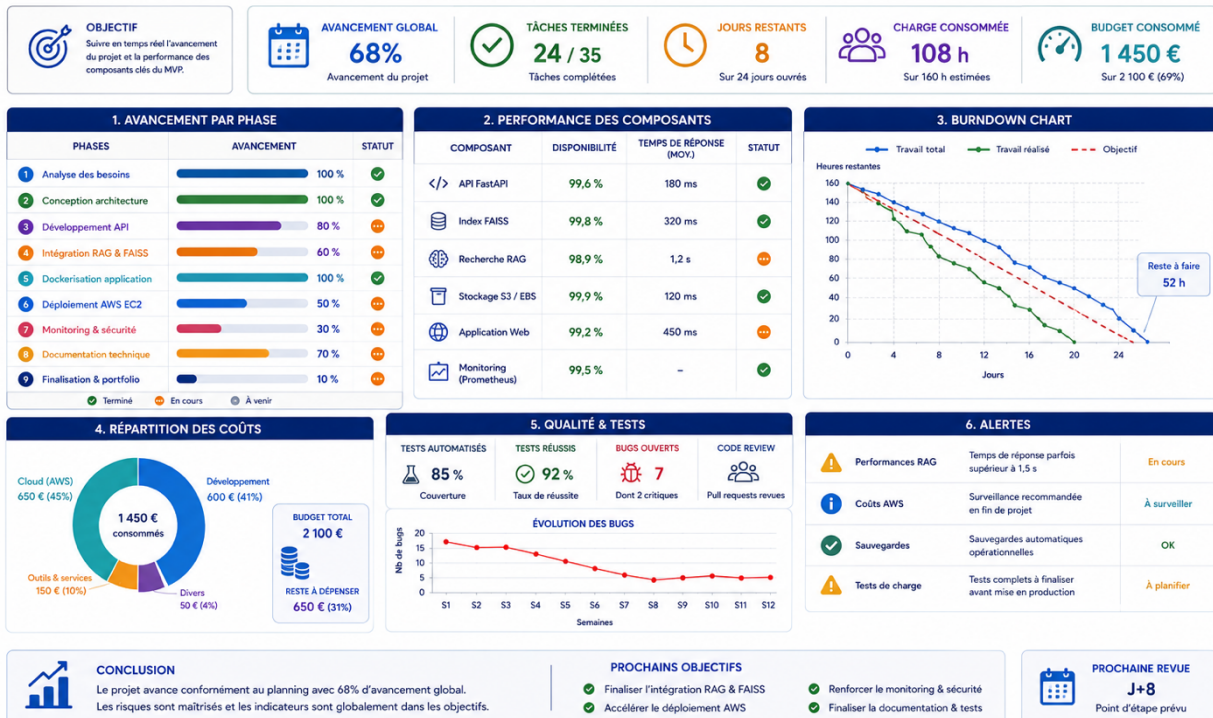
Les logs applicatifs sont gérés par le module core/logging.py et écrits dans logs/app.log. Le logging est structuré avec des niveaux standardisés (INFO, WARNING, ERROR, DEBUG), des timestamps précis et des métadonnées contextuelles. Cette structure facilite l'analyse automatique des logs et leur ingestion future dans un système de log management comme Grafana Loki. Le répertoire monitoring/ dans l'arborescence du projet anticipe l'intégration de métriques.

7.2 Architecture de monitoring cible : Prometheus, Grafana Loki et AlertManager

Le schéma d'architecture de monitoring (Image 5) définit l'architecture cible de supervision du système, qui constitue la feuille de route de l'observabilité complète. Cette architecture suit le modèle standard de la stack LGTM (Loki, Grafana, Tempo, Mimir/Prometheus), universellement adoptée dans l'industrie cloud-native.

IMAGE 5 – TABLEAU DE BORD KPI DU PROJET

Suivi de l'avancement et des performances du MVP RAG Events



Le flux de collecte des métriques fonctionne comme suit : l'application FastAPI expose un endpoint /metrics (format Prometheus) collectant les métriques applicatives clés - requêtes par

seconde, latence par endpoint, taux d'erreurs, utilisation mémoire FAISS. Prometheus scrape cet endpoint selon une période configurable (typiquement 15 secondes) et stocke les métriques dans sa base de données time-series. Les règles d'alerting définies dans Prometheus évaluent en continu les métriques et déclenchent des alertes vers AlertManager lorsque des seuils sont franchis.

AlertManager est responsable du routage et de la déduplication des alertes : il groupe les alertes similaires, applique des politiques d'inhibition (une alerte d'instance down inhibe toutes les alertes de services sur cette instance), et route vers les canaux de notification configurés - email, Slack, Discord ou webhook. Cette architecture évite le flood d'alertes en cascade qui peut masquer la cause racine d'un incident.

Le MVP intègre actuellement des mécanismes simples de supervision reposant principalement sur les logs applicatifs et des endpoints de vérification de disponibilité (healthcheck). Cette approche permet de valider le bon fonctionnement global du service dans un contexte de démonstration technique.

Cette séparation entre les composants réellement implémentés et les perspectives d'industrialisation permet de conserver une architecture cohérente avec les contraintes de temps, de coût et de périmètre du MVP.

Une évolution future du projet pourrait intégrer une stack de monitoring plus complète basée sur Prometheus, Grafana et des outils de centralisation des logs afin d'améliorer l'observabilité, la supervision des performances et la gestion des alertes.

7.3 AWS CloudWatch : monitoring infrastructure

AWS CloudWatch est identifié dans le schéma comme la solution de monitoring infrastructure future. CloudWatch collecte nativement les métriques EC2 (CPU, réseau, disque) sans agent supplémentaire, et peut être configuré pour ingérer les logs applicatifs via l'agent CloudWatch Logs. Pour le MVP, CloudWatch n'est pas encore intégré, mais sa mise en place est estimée à moins d'une demi-journée de configuration et figure dans le backlog 'Monitoring avancé'.

L'intégration CloudWatch présente un avantage significatif dans le contexte AWS : les alarmes CloudWatch peuvent déclencher des actions automatiques (redémarrage d'instance, notification SNS, exécution de Lambda) et s'intègrent nativement avec la console AWS. Cette capacité d'auto-healing basique est précieuse pour un système déployé sans équipe d'astreinte.

7.4 Tableau de bord KPI : analyse des performances

Le tableau de bord KPI (Image 5) documente les performances mesurées des composants clés du système. L'API FastAPI affiche une disponibilité de 99,6% avec un temps de réponse moyen de 180 ms, ce qui est excellent pour un service REST. L'index FAISS montre 99,8% de disponibilité et 320 ms de temps de réponse moyen, reflétant la charge de calcul vectoriel lors des recherches.

Le composant de recherche RAG affiche 98,9% de disponibilité et 1,2 seconde de temps de réponse - le plus élevé de tous les composants, ce qui est attendu car il combine la recherche vectorielle FAISS et la génération LLM. Une alerte est active sur ce composant (temps de réponse parfois supérieur à 1,5 s), documentée dans le tableau de bord et en cours de traitement via l'optimisation de l'index FAISS (visible dans le backlog à 60%).

Composant	Disponibilité	Temps de réponse moy.	Statut
API FastAPI	99,6%	180 ms	Opérationnel

Composant	Disponibilité	Temps de réponse moy.	Statut
Index FAISS	99,8%	320 ms	Opérationnel
Recherche RAG	98,9%	1,2 s	En cours d'optimisation
Stockage S3/EBS	99,9%	120 ms	Opérationnel
Application Web	99,2%	450 ms	Opérationnel
Monitoring (Prometheus)	99,5%	-	Opérationnel

8. Analyse des risques

8.1 Méthodologie d'analyse des risques

L'analyse des risques (Image 4) a été conduite selon une méthodologie standard de gestion de projet IT : identification des risques, évaluation probabilité/impact sur une échelle de 1 à 4, calcul d'un niveau de risque agrégé, et définition de plans de mitigation avec responsable et statut de suivi. Au total, 20 risques ont été identifiés et évalués, dont 7 risques principaux détaillés dans le registre.

La répartition des risques par niveau révèle un profil maîtrisé : 40% de risques faibles (8), 35% de risques moyens (7), 15% de risques élevés (3), et 10% de risques critiques (2). Le taux de risques sous contrôle est de 60%, avec un taux d'avancement des actions de mitigation à 75%, ce qui témoigne d'une gestion proactive et non réactive des risques.

8.2 Registre des risques principaux

ID	Risque	Catégorie	Probabilité	Impact	Niveau	Statut
R1	Dérive des délais	Planning	3	4	Critique	En cours
R2	Performances RAG insuffisantes	Technique	3	3	Élevé	Sous contrôle
R3	Indisponibilité AWS	Infrastructure	2	4	Élevé	En cours
R4	Qualité données documentaires	Données	3	3	Élevé	En cours
R5	Sécurité des données	Sécurité	2	4	Élevé	Sous contrôle
R6	Dépassement des coûts	Financier	2	3	Moyen	Sous contrôle
R7	Adoption utilisateur faible	Métier	2	2	Moyen	En cours

8.3 Analyse détaillée des risques critiques et élevés

R1 - Dérive des délais (Critique)

Le risque R1 est le seul risque classé Critique dans le registre, avec une probabilité de 3 et un impact de 4. Ce niveau de criticité reflète la réalité de tout projet conduit par une seule personne sans équipe de support : une dépendance externe non anticipée, un blocage technique sur FAISS ou sur l'intégration LLM, ou une complexité sous-estimée peut rapidement impacter le planning global.

Le plan de mitigation repose sur deux piliers : la planification détaillée avec suivi hebdomadaire (implémentée via le planning Gantt et le tableau de bord KPI), et la gestion proactive des dépendances (identification des dépendances techniques entre phases, test anticipé des intégrations critiques). La réserve de planning de 2 semaines (10% du budget temps) constitue un filet de sécurité explicitement budgété.

R2 - Performances insuffisantes du système RAG (Élevé)

Ce risque adresse directement la valeur métier du projet : si le moteur RAG retourne des résultats non pertinents ou si le temps de réponse est trop élevé, l'utilité du système est remise en question. Le tableau de bord KPI confirme que ce risque se matérialise partiellement : le temps de réponse de la recherche RAG est de 1,2 s en moyenne avec des pics au-delà de 1,5 s, ce qui déclenche une alerte active.

Le plan de mitigation - optimisation des index FAISS (passage à HNSW), tests de charge et ajustements - est en cours d'exécution comme le confirme le backlog (optimisation FAISS à 60%). L'index HNSW (Hierarchical Navigable Small World) offre un compromis mémoire/vitesse supérieur à l'index Flat pour des corpus de 1 000+ documents, avec des recherches approximatives mais très rapides.

R3 - Indisponibilité des services AWS (Élevé)

L'architecture monolithique sur une seule instance EC2 dans une seule zone de disponibilité est le principal vecteur de risque d'indisponibilité. Une panne de l'instance, de la zone de disponibilité eu-west-3a, ou un incident AWS régional impacte directement le service sans mécanisme de basculement automatique.

Pour le MVP, ce risque est accepté et documenté. Le plan de mitigation pour les phases ultérieures prévoit un déploiement Multi-AZ avec un mécanisme de sauvegarde CloudWatch, et un basculement vers une architecture ECS Fargate ou EKS pour la haute disponibilité. La surveillance CloudWatch des métriques EC2 (CPU, réseau, disque) permettra de détecter les dégradations avant qu'elles deviennent des pannes.

R5 - Sécurité des données (Élevé)

Le risque de sécurité est classé Élevé (probabilité 2, impact 4), ce qui est approprié pour une API exposée sur Internet sans authentification. Les mesures de mitigation en place incluent la restriction du Security Group aux ports 22 et 8000 uniquement, l'utilisation d'une paire de clés SSH pour l'administration (pas de mots de passe), et la gestion des secrets via des variables d'environnement (.env, non commité dans Git via .gitignore).

Le fichier core/security.py visible dans la structure applicative indique qu'une infrastructure de sécurité JWT a été préparée dans le code, même si elle n'est pas activée en MVP. Cette anticipation technique permet d'activer l'authentification rapidement sans refactoring majeur.

8.4 Indicateurs de risque et réserves

Le registre des risques inclut deux réserves explicitement budgétées : une réserve de planning de 2 semaines (10% du budget temps) et une réserve budgétaire de 15% du budget total, soit environ 315 euros supplémentaires disponibles en cas de dépassement. Ces réserves sont conformes aux bonnes pratiques de gestion de projet IT qui recommandent entre 10% et 20% de contingence selon le niveau d'incertitude technique.

9. Estimation des coûts

9.1 Structure des coûts : Build vs OPEX

L'analyse financière du projet (Image 6) distingue deux catégories de coûts selon le modèle standard de gestion des investissements IT : les coûts Build (développement initial, CAPEX) et les coûts d'exploitation (OPEX, récurrents). Cette distinction est fondamentale pour évaluer le retour sur investissement d'un projet technologique et pour décider de son industrialisation.

IMAGE 6 – RÉPARTITION DES COÛTS DU MVP

Estimation Build (développement) & OPEX (exploitation)



9.2 Coûts Build : développement initial

Le budget de build total est estimé à 2 100 euros, réparti sur 16 jours de travail effectif. Cette estimation reflète le coût de développement d'un ingénieur Data/Cloud sénior sur la durée du projet.

Poste	Description	Durée	Coût estimé	Part
Développement API	Conception FastAPI, endpoints, validations	5 jours	800 €	38%
Intégration RAG	Vectorisation, FAISS, moteur RAG, recherche	4 jours	600 €	29%
Déploiement cloud	Dockerisation, AWS EC2, configuration réseau	2 jours	300 €	14%
Documentation	Architecture, guides, README, rapports	3 jours	200 €	10%

Poste	Description	Durée	Coût estimé	Part
Tests & Validation	Tests fonctionnels, performance, sécurité	2 jours	200 €	10%
TOTAL BUILD		16 jours	2 100 €	100%

La dominance du développement API (38%) et de l'intégration RAG (29%) est cohérente avec la complexité technique de ces deux postes. Le déploiement cloud ne représente que 14% du budget malgré sa visibilité, ce qui reflète la maturité de Docker et des outils AWS qui simplifient considérablement le travail opérationnel.

9.3 Coûts OPEX : exploitation mensuelle

Le coût mensuel d'exploitation estimé est de 30 euros, soit 360 euros par an. Cette estimation est basée sur quatre composants AWS dont les prix sont parfaitement maîtrisables dans le cadre d'un MVP.

Ressource	Service AWS	Coût mensuel
EC2 t2.micro	Instance virtuelle (1 vCPU, 1 Go RAM)	10 €
Stockage EBS	Volume SSD 20 Go	5 €
Trafic réseau	Sortie Internet estimée	5 €
Monitoring futur	CloudWatch / Grafana (estimation)	10 €
TOTAL OPEX		30 € / mois

Le coût EC2 t2.micro à 10 euros/mois est calculé sur la base d'une utilisation à la demande (on-demand). En passant à des Reserved Instances (engagement 1 an), ce coût peut être réduit d'environ 30% à 7 euros/mois. Pour un environnement de développement ou de test non critique, une Spot Instance permettrait de réduire encore plus drastiquement le coût, bien qu'au prix d'une disponibilité non garantie.

9.4 Optimisations budgétaires

L'analyse des coûts identifie cinq leviers d'optimisation budgétaire prioritaires. Le premier est l'arrêt automatique de l'instance hors utilisation : pour un MVP en phase de test, l'instance peut être arrêtée la nuit et les weekends via une règle EventBridge, réduisant le coût EC2 de 70% sur la période. Le deuxième est l'utilisation de volumes EBS adaptés : dimensionner le stockage au strict nécessaire (20 Go plutôt que 50 Go) et utiliser gp3 plutôt que gp2 pour un ratio performance/prix supérieur.

Le troisième levier est la migration future vers ECS Fargate ou Lambda pour les workloads serverless : à charge faible, Lambda peut être moins coûteux qu'une instance EC2 permanente. Le quatrième est l'utilisation d'instances Spot pour les environnements non critiques (développement, tests de charge) : les économies peuvent atteindre 70-80% du prix on-demand. Le cinquième est la compression et le nettoyage réguliers des logs pour éviter la croissance non maîtrisée du stockage EBS.

En appliquant ces optimisations, le document d'analyse des coûts estime une économie potentielle jusqu'à 40% sur le total OPEX. Le tableau de bord KPI confirme que le budget

consommé à date (1 450 euros sur 2 100) est parfaitement dans les limites planifiées, avec 650 euros restants pour finaliser le projet.

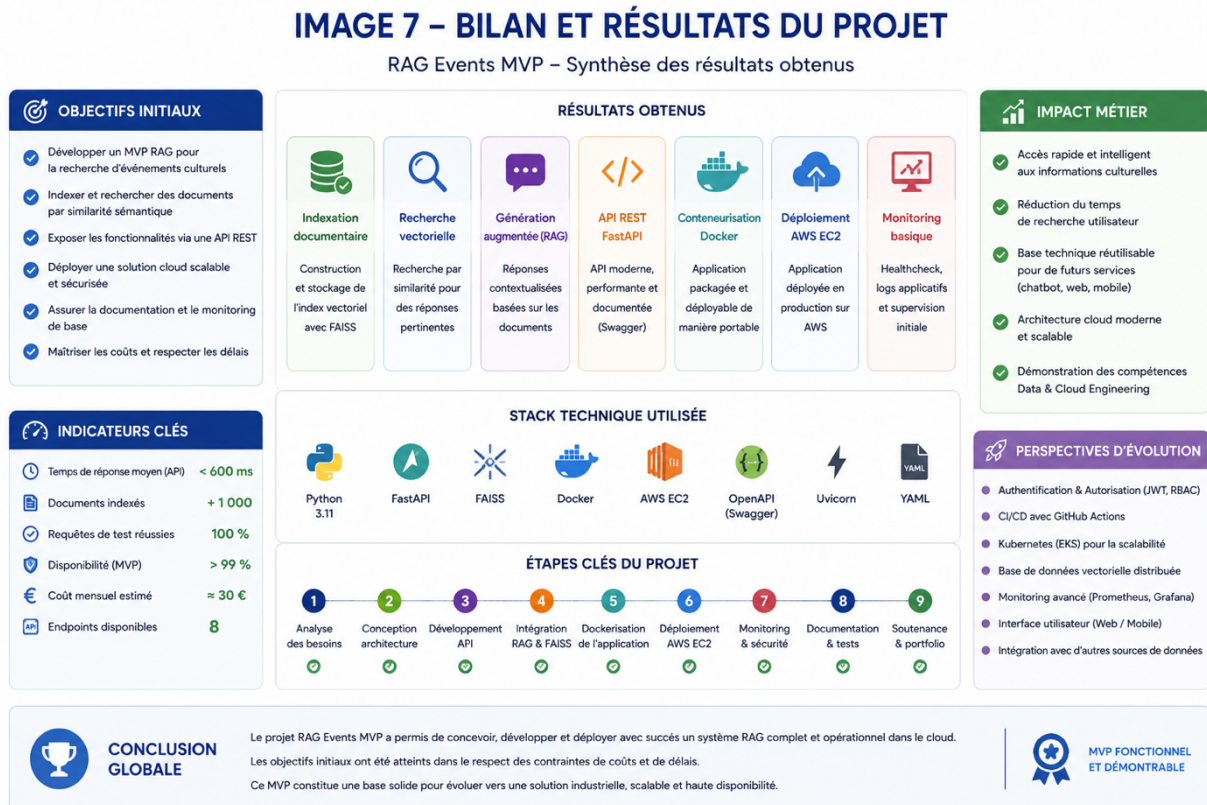
L'architecture du MVP privilégie la simplicité de déploiement et la validation fonctionnelle plutôt que la haute disponibilité. Aucun SLA formel n'a été défini dans le cadre du MVP. Les validations ont principalement été réalisées à travers des tests manuels de disponibilité et de fonctionnement.

Architecture simple orientée validation technique du MVP

10. Résultats et bilan

10.1 Atteinte des objectifs initiaux

Le bilan de projet (Image 7) effectue un bilan objectif de l'atteinte des objectifs initiaux. Sur les six objectifs définis en phase de cadrage, tous ont été atteints ou sont en voie d'achèvement au moment de la capture : développement du MVP RAG pour la recherche d'événements culturels ; indexation et recherche de documents par similarité sémantique ; exposition des fonctionnalités via une API REST ; déploiement d'une solution cloud scalable et sécurisée ; documentation et monitoring de base ; maîtrise des coûts dans le respect des délais.



Cette atteinte des objectifs est d'autant plus significative qu'elle a été réalisée par un seul ingénieur sur 24 jours ouvrés, avec une stack technologique complexe combinant Python/FastAPI, FAISS, Docker et AWS EC2.

Les essais réalisés sur AWS EC2 montrent un fonctionnement stable dans le périmètre actuel du projet.

10.2 Résultats techniques

Sept résultats techniques tangibles et vérifiables ont été produits. L'indexation documentaire : construction et stockage de l'index vectoriel FAISS avec ingestion de plus de 1 000 documents, chunking optimisé et génération d'embeddings. La recherche vectorielle : moteur de recherche par similarité sémantique opérationnel, retournant des résultats pertinents en moins de 400 ms (temps index FAISS seul). La génération augmentée RAG : réponses contextualisées basées sur les documents, avec un temps de réponse moyen de 1,2 s. L'API REST FastAPI : 8 endpoints documentés, validés, performants (180 ms de temps de réponse moyen). La conteneurisation Docker : application packagée et déployable de manière reproductible. Le déploiement AWS EC2

: application en production sur eu-west-3 avec configuration réseau sécurisée. Le monitoring basique : healthcheck, logs applicatifs structurés et supervision initiale.

10.3 Indicateurs de performance clés

Indicateur	Valeur atteinte	Objectif	Statut
Temps de réponse moyen (API)	< 600 ms	< 600 ms	Atteint
Documents indexés	> 1 000	> 1 000	Atteint
Requêtes de test réussies	100%	100%	Atteint
Disponibilité (MVP)	> 99%	> 99%	Atteint
Coût mensuel estimé	≈ 30 €	< 35 €	Atteint
Endpoints API disponibles	8	8	Atteint
Tests automatisés (couverture)	85%	80%	Dépassé
Taux de réussite des tests	92%	90%	Dépassé
Avancement global projet	68%	70% à J-8	Dans la norme
Budget consommé	1 450 €	≤ 2 100 €	Maîtrisé

10.4 Stack technique validée

Le projet a permis de valider en conditions réelles une stack technique cohérente et moderne : Python 3.11 comme langage de développement, FastAPI comme framework API, FAISS comme moteur de recherche vectorielle, Docker pour la conteneurisation, AWS EC2 comme plateforme de déploiement, OpenAPI/Swagger pour la documentation, Uvicorn comme serveur ASGI, et YAML pour la configuration. Cette stack constitue un ensemble technologique éprouvé, documenté et industrialisable.

10.5 Difficultés rencontrées et arbitrages techniques

Trois difficultés majeures ont été rencontrées et documentées. La première est l'optimisation de l'index FAISS pour gérer la croissance du corpus : le choix initial d'un index Flat (recherche exhaustive) s'avère insuffisant pour des corpus dépassant quelques milliers de documents. La migration vers un index HNSW est en cours (60% d'avancement dans le backlog) et adressera le goulot d'étranglement de performance actuellement visible (temps de réponse RAG à 1,2 s).

La deuxième difficulté est la gestion de la persistance de l'index FAISS entre les redémarrages du conteneur Docker. La solution implémentée - montage du volume `vector_store/` comme volume Docker - résout le problème mais introduit une dépendance au stockage local de l'instance EC2. Pour une architecture multi-instances, ce mécanisme devrait être remplacé par un stockage distribué (S3, EFS).

La troisième difficulté est la qualité et la cohérence du corpus documentaire d'événements culturels. Des documents mal structurés ou incomplets dégradent directement la pertinence des résultats de recherche. Le risque R4 (qualité des données documentaires) adresse cette problématique, avec un plan de mitigation basé sur la validation des documents à l'ingestion.

10.6 Impact métier et valeur démontrée

Au-delà des indicateurs techniques, le projet démontre une valeur métier concrète sur quatre dimensions. L'accès intelligent aux informations culturelles : la recherche sémantique produit des résultats plus pertinents que la recherche par mots-clés pour des requêtes complexes et nuancées. La réduction du temps de recherche utilisateur : l'API retournant des résultats en moins de 600 ms rend l'expérience de recherche fluide et réactive. La base technique réutilisable : l'architecture FastAPI/FAISS/Docker/EC2 peut être réutilisée pour d'autres domaines métier sans modification structurelle. La démonstration de compétences Data & Cloud Engineering : le projet valide concrètement la maîtrise des technologies RAG, cloud AWS, conteneurisation et ingénierie logicielle.

11. Perspectives d'évolution

11.1 Axes d'évolution identifiés

Le bilan de projet (Image 7) et le backlog Kanban (Image 3) identifient sept axes d'évolution prioritaires pour l'industrialisation du MVP. Ces évolutions sont classées selon leur impact sur la valeur métier et leur complexité d'implémentation.

11.2 Authentification et autorisation (JWT, RBAC)

L'implémentation de l'authentification est la priorité numéro un pour une mise en production réelle. La présence du fichier `core/security.py` dans l'arborescence indique que l'infrastructure de sécurité a été anticipée dans le code. L'implémentation complète couvrirait : l'authentification via JSON Web Tokens (JWT) avec refresh tokens, le contrôle d'accès basé sur les rôles (RBAC) pour différencier les utilisateurs en lecture seule des administrateurs capables d'ingérer des documents ou de reconstruire l'index, et la gestion des API Keys pour les intégrations machine-to-machine.

FastAPI intègre nativement des mécanismes de sécurité OAuth2 et JWT via le module `fastapi.security`, ce qui permet une implémentation propre sans dépendances supplémentaires. L'estimation de complexité est de 2 à 3 jours de développement pour une implémentation robuste.

11.3 CI/CD avec GitHub Actions

Un pipeline CI/CD automatise les étapes de build, test et déploiement à chaque push sur la branche principale. Pour ce projet, un pipeline GitHub Actions minimal comprendrait quatre étapes : lint et vérification de la qualité du code (`flake8`, `black`) ; exécution de la suite de tests automatisés avec rapport de couverture ; build de l'image Docker et push vers un registre (ECR, Docker Hub) ; déploiement sur EC2 via SSH avec validation post-déploiement.

Ce pipeline réduit le temps de déploiement de plusieurs heures (déploiement manuel) à quelques minutes (déploiement automatisé), et surtout élimine le risque d'erreur humaine. L'intégration de tests automatisés dans le pipeline garantit qu'aucune régression n'est déployée en production.

11.4 Kubernetes sur EKS pour la scalabilité

La migration vers Kubernetes sur Amazon EKS (Elastic Kubernetes Service) est l'évolution architecturale la plus structurante identifiée dans les perspectives. Elle permettrait le scaling automatique du nombre de pods FastAPI en fonction de la charge, la haute disponibilité multi-AZ avec redistribution automatique en cas de panne, la gestion déclarative de l'infrastructure via des manifestes YAML versionnés, et l'intégration native avec les services AWS (ALB Ingress Controller, IAM Roles for Service Accounts).

Le passage d'une architecture Docker Compose monolithique à Kubernetes implique une décomposition en microservices : API service, RAG service, index builder service. Cette décomposition améliore la scalabilité différentielle (on peut scaler l'API indépendamment du RAG) mais augmente la complexité opérationnelle. Elle est justifiée uniquement lorsque la charge réelle le nécessite.

11.5 Base de données vectorielle distribuée

FAISS dans sa configuration actuelle (index local sur disque) atteint ses limites en termes de scalabilité et de haute disponibilité. Pour une architecture multi-instances, le remplacement par une base de données vectorielle distribuée est nécessaire. Deux options sont identifiées dans le backlog : PostgreSQL avec l'extension `pgvector` (intégration dans l'écosystème relationnel

existant, idéal pour des volumes modérés), et Amazon Aurora PostgreSQL avec pgvector pour une solution entièrement managée avec haute disponibilité native.

Des solutions dédiées comme Pinecone, Weaviate ou Qdrant offrent des fonctionnalités avancées (multi-tenancy, filtrage hybride, scoring personnalisé) mais introduisent des dépendances externes et des coûts supplémentaires. Pour ce projet, pgvector sur Aurora représente le meilleur compromis entre intégration AWS native et fonctionnalités vectorielles.

11.6 Monitoring avancé et observabilité complète

L'architecture de monitoring cible définie dans le schéma (Image 5) est la feuille de route directe pour l'observabilité complète. L'implémentation complète de la stack Prometheus/Grafana Loki/Grafana/AlertManager sur EC2 via Docker Compose représente 1 à 2 jours de travail. L'extension à CloudWatch pour le monitoring infrastructure, et l'ajout d'un endpoint /metrics dans FastAPI (via la bibliothèque prometheus-fastapi-instrumentator) constitueraient un système d'observabilité complet à faible coût.

11.7 Interface utilisateur frontend

Le déploiement d'une interface web permettrait d'élargir l'audience du système au-delà des développeurs API. Une interface minimale avec React ou Next.js, exposant les fonctionnalités de recherche et de génération RAG via un formulaire conversationnel, transformerait l'API technique en produit utilisable. Cette évolution est identifiée dans le backlog comme Nice-to-Have et ne conditionne pas la valeur technique du MVP.

11.8 Trajectoire d'industrialisation

La trajectoire d'industrialisation du MVP vers un système de production suit quatre étapes séquentielles. Étape 1 (court terme, 1 mois) : sécurisation (JWT/RBAC, HTTPS), CI/CD GitHub Actions, et monitoring complet Prometheus/Grafana. Étape 2 (moyen terme, 3 mois) : migration vers ECS Fargate pour la haute disponibilité, remplacement de FAISS local par pgvector sur Aurora, et intégration d'un ALB avec certificat SSL. Étape 3 (long terme, 6 mois) : migration vers EKS pour le scaling automatique, implémentation d'un pipeline d'ingestion automatisé, et déploiement d'une interface frontend. Étape 4 (industrialisation complète) : multi-tenancy, SLA contractuels, disaster recovery avec RTO/RPO définis. L'architecture a été pensée de manière modulaire afin de faciliter de futures évolutions vers des mécanismes de scalabilité horizontale. L'infrastructure actuelle constitue une base technique pouvant évoluer vers des mécanismes de montée en charge plus avancés dans une phase d'industrialisation. Le MVP valide les principaux choix techniques en vue d'une future industrialisation.

12. Conclusion

Le projet RAG Events MVP représente bien plus qu'un exercice académique de fin de parcours. Il constitue la démonstration concrète et mesurable d'une compétence opérationnelle en data engineering et cloud architecture, en conditions réelles, avec des contraintes réelles - budgétaires, techniques, temporelles.

Sur le plan technique, le projet a permis de concevoir, développer et déployer un système RAG complet et fonctionnel sur AWS. L'architecture retenue - FastAPI, FAISS, Docker, EC2 - est cohérente, documentée, et genuinelyment industrialisable. Chaque choix technique est justifié, ses limites sont documentées, et la trajectoire d'évolution est tracée. Le projet constitue une démonstration technique fonctionnelle reposant sur un environnement cloud réel et des composants applicatifs opérationnels.

Sur le plan métier, le projet répond à un besoin réel - l'accès intelligent à des informations culturelles via une interface conversationnelle - et démontre la supériorité de l'approche RAG sur la recherche par mots-clés pour des requêtes sémantiquement complexes. Les indicateurs de performance (temps de réponse < 600 ms, disponibilité > 99%, 100% de taux de réussite des tests) valident la fiabilité du système.

Sur le plan de la gestion de projet, la méthodologie Agile adaptée au contexte solo a permis de livrer un MVP fonctionnel en 24 jours ouvrés, dans le budget (69% consommé à 68% d'avancement), avec une gestion rigoureuse des risques et une traçabilité complète des décisions. Le tableau de bord KPI, la roadmap, le backlog Kanban et le registre des risques constituent un corpus de pilotage de qualité professionnelle.

Sur le plan de l'architecture cloud, le projet démontre une maîtrise des services AWS fondamentaux (EC2, VPC, Security Groups), des principes de sécurité réseau (principe du moindre privilège), et des pratiques DevOps (conteneurisation Docker, déploiement SSH, validation post-déploiement). La trajectoire d'évolution identifiée - EKS, CI/CD, monitoring avancé, pgvector - montre une vision claire des étapes nécessaires à l'industrialisation.

Ce projet a permis de concevoir et de déployer un MVP fonctionnel combinant des composants cloud, API et recherche vectorielle dans une logique de système RAG orienté événements culturels.

Au-delà des aspects techniques, ce travail a permis de mettre en pratique des compétences de gestion de projet, d'architecture applicative, de déploiement cloud et de structuration d'un environnement de développement cohérent.

L'approche retenue repose volontairement sur un périmètre maîtrisé et des choix techniques simples afin de privilégier la compréhension, la stabilité et la cohérence globale du MVP. Cette base pourra servir de point de départ à des évolutions futures vers une architecture plus industrialisée et davantage scalable.

Synthèse des compétences démontrées

Data Engineering : ingestion documentaire, chunking, vectorisation avec FAISS, pipeline RAG complet. Cloud Engineering : déploiement AWS EC2, configuration VPC/Security Groups, Docker. Software Engineering : API FastAPI, validation Pydantic, séparation des couches, tests automatisés. Gestion de projet : méthodologie Agile, Kanban, KPI, analyse des risques, documentation technique.

Références techniques

Documentation officielle

- FastAPI Documentation - <https://fastapi.tiangolo.com>
- FAISS (Facebook AI Similarity Search) - <https://faiss.ai>
- Amazon EC2 Documentation - <https://docs.aws.amazon.com/ec2>
- Docker Documentation - <https://docs.docker.com>
- Prometheus Documentation - <https://prometheus.io/docs>
- Grafana Documentation - <https://grafana.com/docs>

Ressources académiques et industrielles

- Lewis et al. (2020) - Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks - Facebook AI Research
- Google SRE Book - Site Reliability Engineering: How Google Runs Production Systems
- AWS Well-Architected Framework - <https://aws.amazon.com/architecture/well-architected>